

MODULE 09

Immune

System

Keep Everything

Working

Watchdog, feedback loops, monitoring and
resilience · what separates hobby from production

DURATION

20 min

FORMAT

Demo + slides

KIT

2 files

MODULE 09 · IMMUNE SYSTEM

Build Your AI COO

The Truth No One Tells

"Agents are 30% of the work. The other 70% is the immune system."

· Eric Siu

You've set up the agent. It's working. But how long until it breaks silently?

The Real Problem

Silent Failures

Crons stop running. Sub-agents crash in limbo. APIs change and you find out days later when someone complains.

· Runaway Costs

A bug makes the agent call the API 1000x per hour. You wake up with a \$500 bill.

· Repetition of Errors

Agent suggests something bad, you reject it. Next session: same suggestion. No memory of feedback = no learning.

The Solution: Immune System

A system that monitors, detects, recovers, and learns automatically. 5 layers:

1. Watchdog · monitors crons and auto-retry
2. Feedback Loops · learns from approve/reject
3. Cost Monitoring · prevents runaway billing
4. Security Audits · periodic hardening
5. Backup & Rollback · revert when things go wrong

·This module separates "just kidding" from "I'm in production"
If you want a reliable agent running 24/7, you NEED immunity.

Module 09 · Immune System

02

MODULE 09 · IMMUNE SYSTEM

Build Your AI COO

1. Watchdog: Monitor and Self-Recover

A cron that monitors the other crons. If something fails, it tries to fix it on its own before alerting you.

Logic

1. List all active crons
2. Check the last run of each
3. If any failed · automatic retry (up to 3x)
4. If it failed 3x · alert on Telegram

Why it Works

isolated + agentTurn = actually executes (doesn't just trigger)

Automatic retry before alerting = less noise

Runs 1x/day = catches problems early without spending much

2. Feedback Loops: Learn from Decisions

The agent needs to remember what you approved and rejected. Without that, it will repeat the same mistakes every session.

Setup

// Cron Watchdog · runs 1x/day

```
{
  "name": "Watchdog - Cron Monitor",
  "schedule": {
    "kind": "cron",
    "expr": "0 8 * * **",
    "tz": "America/Sao_Paulo"
  },
  "sessionTarget": "isolated",
  "payload": {
    "kind": "agentTurn",
    "message": "Check the health of all crons. List those that have failed in the last",
  },
  "delivery": { "mode": "announce" }
}
```

JSON

Create feedback structure

mkdir -p memory/feedback

Files per domain

touch memory/feedback/content.json

BASH

touch memory/feedback/tasks.json

touch memory/feedback/recommendations.json

Módulo 09 · Sistema Immunológico

03

MODULE 09 · IMMUNE SYSTEM

Build Your AI COO

Feedback Format

CRITICAL Rules

·Max 30 entries per file

When it reaches 30, remove the oldest ones (FIFO). Otherwise the context explodes.

·Agent MUST consult before suggesting

Add to AGENTS.md: "Before suggesting something (content, task, tool), consult memory/feedback/ to avoid repeating mistakes."

Consolidation Cycle

Feedback does not replace permanent memory. Every 30 days:

1. Feedback (granular, JSON) · raw data of decisions
2. Lessons (curated, prose) · extracted patterns
3. Decisions (permanent) · consolidated policies

```
{
  "entries": [
    {
      "date": "2026-02-13",
      "context": "Suggested thread about X for LinkedIn",
      "decision": "approve",
      "reason": "Accurate tone, specific data",
      "tags": ["linkedin", "thread", "tone"]
    },
    {
      "date": "2026-02-14",
      "context": "Suggested using skill X for task Y",
      "decision": "reject",
      "reason": "Skill is not reliable, I prefer to do it manually",
      "tags": ["skills", "tools"]
    }
  ]
}
```

JSON

·Example

5 skill rejections · lesson: "Prefers to do X manually" · decision: "Never suggest skill Y for task X"

Module 09 · Immune System

04

MÓDULO 09 · SISTEMA IMUNOLÓGICO

Construa Seu AI COO

3. Monitoramento de Custos

Prevenir runaway billing ANTES de virar problema.

Split de Modelos por Uso

USO

MODELO

CUSTO RELATIVO

Interação direta

Opus

\$\$\$

Crons e automação

Sonnet

\$

Heartbeats

Haiku

¢

Regra INVIOLÁVEL

TODOS os crons devem rodar em Sonnet (nunca Opus). Heartbeats em Haiku. Só a interação direta usa Opus.

Rate Limits e Budgets

Isso previne loops infinitos e bugs que drenam a API.

Economia Real

Tudo em Opus

$17 \text{ crons/dia} \times \$0.10 = \$1.70/\text{dia}$

$\text{Heartbeats } 4x/\text{dia} \times \$0.10 = \$0.40/\text{dia}$

Interação = $\sim \$0.50/\text{dia}$

· Split otimizado

$17 \text{ crons/dia} \times \$0.01 = \$0.17/\text{dia}$

$\text{Heartbeats } 4x/\text{dia} \times \$0.005 = \$0.02/\text{dia}$

Interação = $\sim \$0.50/\text{dia}$

// Em openclaw.json

```
{  
  "model": {  
    "default": "anthropic/claude-opus-4-6",  
    "budget": {  
      "daily_max_usd": 5.0,  
      "alert_threshold_usd": 3.0  
    },  
    "rate_limit": {  
      "max_requests_per_minute": 10  
    }  
  }  
}
```

JSON

Total: ~\$2.60/day

Total: ~\$0.70/day

73% savings just by having the right model in the right place.

Module 09 · Immune System

05

MODULE 09 · IMMUNE SYSTEM

Build Your AI COO

4. Periodic Security Audit

Security degrades over time. Doors that closed can open. Fail2ban can deactivate. SSH can degrade.

Weekly Audit Cron

Audit Checklist

- UFW active and configured correctly
- Fail2ban running with SSH rules
- Exposed ports = minimum necessary
- SSH: PasswordAuthentication no, PermitRootLogin no
- Credentials in 1Password (NEVER hardcoded)
- Valid SSL certificates (if applicable)
- Packages updated (apt update + upgrade)

Auto-Fix vs Manual

- Never auto-fix security without review

The agent should REPORT issues and SUGGEST corrections. You approve. Otherwise, risk of lockout.

5. Backup and Rollback

Before any structural change (config, new agents, reorganization), save a backup + rollback instructions.

```
{
  "name": "Security Audit - Weekly",
  "schedule": {
    "kind": "cron",
    "expr": "0 6 * * 0",
    "tz": "America/Sao_Paulo"
  },
  "sessionTarget": "isolated",
  "payload": {
    "kind": "agentTurn",
    "message": "Run complete security audit. Check: UFW active, Fail2ban runs"
  },
  "delivery": { "mode": "announce" }
}
```

JSON

```
# Backup before change
mkdir -p backups/$(date +%Y-%m-%d)
cp /root/.openclaw/openclaw.json backups/$(date +%Y-%m-%d)/
cp -r memory/ backups/$(date +%Y-%m-%d)/
BASH
Module 09 · Immune System
06
```

MODULE 09 · IMMUNE SYSTEM

Build Your AI COO

ROLLBACK.md

Always leave instructions on how to revert:

Sub-Agents: Never "Fire and Forget"

Every sub-agent needs follow-up. Without it, it can get stuck in limbo and you never know.

Mandatory Rule

1. When spawning: inform what it's going to do
2. Follow-up in 15-30min: check status
3. Success: summarize result
4. Failure: immediate retry · if it fails 2x · warn
5. NEVER let it fall into silent limbo

Real Case: Sub-Agent in Limbo

I spawned an agent to generate a report. It failed silently. I found out 3 days later when I looked for the result. Solution: always follow-up.

Ralph Loop vs Feedback Loop

Two different concepts:

ROLLBACK.md

Backup Date: 2026-02-16

What Changed:

- Created "content-curator" agent
- Modified model split (Opus · Sonnet in crons)

How to Revert:

1. Restore config:

```
cp backups/2026-02-16/openclaw.json /root/.openclaw/
```

```
openclaw gateway restart
```

2. Delete created agent:

```
rm -rf agents/content-curator/
```

3. Verify:

```
openclaw gateway status
```

MARKDOWN

LOOP

PURPOSE

Ralph Loop

Coding loop: write code · test · fix · repeat (1 session)

Feedback

Loop

Learning between sessions: you approve/reject · agent remembers forever

Module 09 · Immune System

07

MODULE 09 · IMMUNE SYSTEM

Build Your AI COO

· Module 9 Checkpoint

Cron watchdog configured and active

Feedback loops created (at least 1 domain)

Model split applied (Opus/Sonnet/Haiku)

Rate limits and budgets configured

Weekly security audit scheduled

Backup rule before changes documented

Sub-agents with mandatory follow-up in AGENTS.md

· Prompt for the Agent

Paste this prompt into your OpenClaw chat after completing the module:

I just watched Module 9 on the immune system. "Agents are 30% of the work. The other 70% is the immune system." Guide me. What I need you to do: 1.

Cron watchdog · Configure a cron that monitors if the other crons are running. If any fail, automatic retry up to 3x. If it fails 3x, notify me. 2.

Feedback Loops · Help me set up an approve/reject system. When you suggest something to me and I reject it, note the reason. Consult these notes before suggesting again. Show me how this works in practice. 3. Cost monitoring ·

Configure the model split (Haiku for heartbeats, Sonnet for crons, Opus for interaction). Configure rate limits and budgets to prevent runaway. Show me how much I'm spending per day/week. Goal: from ~\$2-3/day to ~\$0.10/day with optimizations. 4. Periodic security audit · Configure a weekly cron for

security audit. Run one now for me to see the result. 5. Backup before changes ·

Create a rule: before any structural change, save backup + ROLLBACK.md.

Show me how to revert if something goes wrong. Rules: - This module is what separates "I'm playing around" from "I'm in production" - Explain each protection and what problem it prevents - At the end, run a complete health check and give me the score Let's build the immunity?

NEXT MODULE

..Module 10 · Mission Control: Your Operational Dashboard

Module 09 · Immune System

08